Grokking as the Transition from Lazy to Rich Training Dynamics

Exercise Session - 1h, pen and paper

Reference: Kumar, Bordelon, Gershman & Pehlevan, ICLR 2024

Problem 1 - The Model and its Summary Statistics (10 min)

Consider a two-layer committee machine with N hidden neurons, input $\boldsymbol{x} \in \mathbb{R}^D$, and fixed readout weights all equal to 1:

$$f(\boldsymbol{w}, \boldsymbol{x}) = \frac{\alpha}{N} \sum_{i=1}^{N} \varphi(w_i \cdot \boldsymbol{x}), \qquad \varphi(h) = h + \frac{\epsilon}{2} h^2$$

The target function is $y(\boldsymbol{x}) = \frac{1}{2}(\boldsymbol{\beta}_\star \cdot \boldsymbol{x})^2$, with $\boldsymbol{x} \sim \mathcal{N}(0, \frac{1}{D}\boldsymbol{I})$.

(a) Define the two summary statistics:

$$\bar{\boldsymbol{w}} = \frac{1}{N} \sum_{i=1}^{N} \boldsymbol{w}_i, \qquad \boldsymbol{M} = \frac{1}{N} \sum_{i=1}^{N} \boldsymbol{w}_i \boldsymbol{w}_i^\top$$

Show that the network output can be written as:

$$f(\boldsymbol{x}) = \alpha\, \bar{\boldsymbol{w}} \cdot \boldsymbol{x} \; + \; \frac{\alpha\epsilon}{2}\, \boldsymbol{x}^\top \boldsymbol{M}\, \boldsymbol{x}$$

(b) What are the values of $\bar{\boldsymbol{w}}$ and $\boldsymbol{M}$ at random initialisation in the large N limit?

(c) For the network to perfectly fit the target on the test distribution, what must $\boldsymbol{M}$ and $\bar{\boldsymbol{w}}$ equal? (*Hint*: match the quadratic and linear parts of f to y.)

Solution

(a) Expand $\varphi(w_i \cdot x) = w_i \cdot x + \frac{\epsilon}{2}(w_i \cdot x)^2$. Sum over i:

$$f = \frac{\alpha}{N} \sum_i \left[w_i \cdot x + \frac{\epsilon}{2}(w_i \cdot x)^2 \right] = \alpha \left(\frac{1}{N} \sum_i w_i \right) \cdot x + \frac{\alpha\epsilon}{2}\, x^\top \left(\frac{1}{N} \sum_i w_i w_i^\top \right) x = \alpha\, \bar{w} \cdot x + \frac{\alpha\epsilon}{2}\, x^\top M\, x$$

(b) At random initialisation with i.i.d. w_i: by the law of large numbers, $\bar{\boldsymbol{w}} \to \mathbb{E}[w_i] = \boldsymbol{0}$ and $\boldsymbol{M} \to \mathbb{E}[w_i w_i^\top] = \boldsymbol{I}$ (assuming standard normal init with variance 1 per component).

(c) The target is $y = \frac{1}{2}(\beta_\star \cdot x)^2 = \frac{1}{2}x^\top \beta_\star \beta_\star^\top x$. Matching the quadratic part: $\frac{\alpha\epsilon}{2} M = \frac{1}{2}\beta_\star \beta_\star^\top$, so $\boldsymbol{M}^\star = \frac{1}{\alpha\epsilon}\boldsymbol{\beta}_\star \boldsymbol{\beta}_\star^\top$. The target has no linear component, so $\bar{\boldsymbol{w}}^\star = \boldsymbol{0}$.

Problem 2 - The NTK of the Toy Model (10 min)

(a) Compute the NTK $K(\boldsymbol{x}, \boldsymbol{x}') = \sum_{i=1}^N \nabla_{w_i} f(\boldsymbol{x}) \cdot \nabla_{w_i} f(\boldsymbol{x}')$ for this model. Show that, up to the overall prefactor α^2/N coming from this parametrisation, it can be expressed in terms of $\bar{\boldsymbol{w}}$ and $\boldsymbol{M}$ as:

$$K(\boldsymbol{x}, \boldsymbol{x}') \propto (\boldsymbol{x} \cdot \boldsymbol{x}') + \epsilon\, (\boldsymbol{x} \cdot \boldsymbol{x}')\, \bar{\boldsymbol{w}} \cdot (\boldsymbol{x} + \boldsymbol{x}') + \epsilon^2\, (\boldsymbol{x} \cdot \boldsymbol{x}')\, \boldsymbol{x}^\top \boldsymbol{M}\, \boldsymbol{x}'$$

(b) At initialisation ($\bar{\boldsymbol{w}} = 0$, $\boldsymbol{M} = \boldsymbol{I}$), the kernel simplifies to

$$K_0(\boldsymbol{x}, \boldsymbol{x}') \propto (\boldsymbol{x} \cdot \boldsymbol{x}') + \epsilon^2 (\boldsymbol{x} \cdot \boldsymbol{x}')^2.$$

This kernel is a sum of two terms. The first term $(\boldsymbol{x} \cdot \boldsymbol{x}')$ is sensitive to *linear* structure in the data, and the second $\epsilon^2(\boldsymbol{x} \cdot \boldsymbol{x}')^2$ to *quadratic* structure. Compute the ratio of the typical magnitude of the quadratic term to the linear term when x, x' are independent draws from $\mathcal{N}(0, \frac{1}{D} I)$. What does this tell you about how well-suited the initial kernel is for learning the (purely quadratic) target, especially when ϵ is small?

(*Hint*: use $\mathbb{E}[(\boldsymbol{x} \cdot \boldsymbol{x}')^2]$ for typical magnitude)

(c) The target $y(x) = \frac{1}{2}(\beta_\star \cdot x)^2$ is purely quadratic. After feature learning, the network can align $\boldsymbol{M}$ with $\beta_\star \beta_\star^\top$, effectively reducing the problem to learning a single direction in $\mathbb{R}^D$. Argue intuitively that this requires $P \sim D$ samples. By contrast, a fixed kernel that treats all quadratic directions equally would need far more samples. Why does this create an opportunity for grokking?

Solution 2

(a) We have $\nabla_{w_i} f = \frac{\alpha}{N} \varphi'(w_i \cdot x)\, x = \frac{\alpha}{N}(1 + \epsilon\, w_i \cdot x)\, x$. Therefore:

$$K(x, x') = \sum_i \nabla_{w_i} f(x) \cdot \nabla_{w_i} f(x') = \frac{\alpha^2}{N^2} \sum_i (1 + \epsilon\, w_i \cdot x)(1 + \epsilon\, w_i \cdot x')\, (x \cdot x')$$

Expanding gives

$$K(x, x') = \frac{\alpha^2}{N} (x \cdot x') \left[1 + \epsilon\, \bar{w} \cdot (x + x') + \epsilon^2\, x^\top M\, x'\right].$$

So, after dividing out the overall prefactor α^2/N, we obtain

$$(x \cdot x') + \epsilon(x \cdot x')\bar{w} \cdot (x + x') + \epsilon^2(x \cdot x')\, x^\top M\, x'.$$

(b) The linear term has typical magnitude $\mathbb{E}[|x \cdot x'|] \sim \sqrt{\mathbb{E}[(x \cdot x')^2]} = 1/\sqrt{D}$. The quadratic term has typical magnitude $\epsilon^2\, \mathbb{E}[(x \cdot x')^2] = \epsilon^2/D$. The ratio is:

$$\frac{\text{quadratic}}{\text{linear}} \sim \frac{\epsilon^2/D}{1/\sqrt{D}} = \frac{\epsilon^2}{\sqrt{D}}$$

For large D and fixed ϵ, this ratio is small: the initial kernel is dominated by its sensitivity to linear structure. Since the target is purely quadratic, the initial kernel is badly suited for the task. When ϵ is small, the mismatch is even worse.

(c) After feature learning, the network concentrates M onto the single direction $\beta_\star$. Identifying one direction in $\mathbb{R}^D$ requires $P \sim D$ samples (you need roughly D independent equations to pin down D unknowns). Without feature learning, the fixed kernel must resolve all $\sim D^2/2$ independent quadratic directions $(x_i x_j)$ equally, since $M = I$ treats them all the same - this requires $P \sim D^2$ samples. The gap creates a *Goldilocks zone* $D \ll P \ll D^2$: enough data to generalise *if* features are learned, but not enough for the kernel method. A network that starts lazy will memorise via the kernel, then - once feature learning kicks in - will suddenly generalise. This delay is grokking.

Problem 3 - Loss Decomposition (15 min)

The test MSE of the model is $\mathcal{L} = \langle (y - f)^2 \rangle$ where $\langle \cdot \rangle$ is the expectation over $x \sim \mathcal{N}(0, \frac{1}{D} I)$.

Useful identity (Isserlis' theorem for Gaussians). For $x \sim \mathcal{N}(0, \frac{1}{D} I)$ and any symmetric matrices A, B:

$$\langle (x^\top A\, x)(x^\top B\, x) \rangle = \frac{1}{D^2}\, [(\operatorname{Tr} A)(\operatorname{Tr} B) + 2 \operatorname{Tr}(AB)]$$

In particular, $\langle (x^\top A\, x)^2 \rangle = \frac{1}{D^2}[(\operatorname{Tr} A)^2 + 2|A|_F^2]$ where $|A|_F^2 = \operatorname{Tr}(A^\top A)$.

(a) Decompose the network output as $f = f_{\text{lin}} + f_{\text{quad}}$ with $f_{\text{lin}} = \alpha \bar{\boldsymbol{w}} \cdot \boldsymbol{x}$ and $f_{\text{quad}} = \frac{\alpha\epsilon}{2} \boldsymbol{x}^\top \boldsymbol{M}\, \boldsymbol{x}$. Note that the target $y = \frac{1}{2} \boldsymbol{x}^\top \boldsymbol{\beta}_\star \boldsymbol{\beta}_\star^\top \boldsymbol{x}$ is purely quadratic. Use the fact that odd moments of isotropic Gaussians vanish (i.e. $\langle x_i\, g(x^\top A x) \rangle = 0$) to argue that the MSE splits as:

$$\mathcal{L} = \langle (y - f_{\text{quad}})^2 \rangle + \langle f_{\text{lin}}^2 \rangle$$

Then compute $\langle f_{\text{lin}}^2 \rangle$ explicitly. This is **Term C** (linear power).

(b) Now focus on the quadratic part. Define $A = \boldsymbol{\beta}_\star \boldsymbol{\beta}_\star^\top - \alpha\epsilon\, \boldsymbol{M}$, so that $y - f_{\text{quad}} = \frac{1}{2}\boldsymbol{x}^\top A\, \boldsymbol{x}$. Apply the Isserlis identity above to compute $\langle (y - f_{\text{quad}})^2 \rangle = \frac{1}{4}\langle (\boldsymbol{x}^\top A\, \boldsymbol{x})^2 \rangle$. Show that the result can be written as the sum of two terms:

$$\underbrace{\frac{1}{4D^2}(\operatorname{Tr} A)^2}_{\text{Term A: variance error}} \quad + \quad \underbrace{\frac{1}{2D^2}|A|_F^2}_{\text{Term B: misalignment error}}$$

Write out Term A and Term B explicitly in terms of $|\boldsymbol{\beta}_\star|^2$, $\operatorname{Tr} \boldsymbol{M}$, α, ϵ, and $|\alpha\epsilon \boldsymbol{M} - \boldsymbol{\beta}_\star \boldsymbol{\beta}_\star^\top|_F$.

(c) Interpret each of the three terms (A, B, C) in one sentence.

(d) At initialisation ($\bar{\boldsymbol{w}} = 0$, $\boldsymbol{M} = \boldsymbol{I}$), which terms are large and which are small? What happens in early training: recalling from Problem 2(b) that the initial kernel is much more sensitive to linear structure than quadratic structure, which component of f does the network use first to reduce train loss, and what does this do to the three terms?

Solution 3

(a) We have $y - f = (y - f_{\text{quad}}) - f_{\text{lin}}$. Expanding the square:

$$\mathcal{L} = \langle (y - f_{\text{quad}})^2 \rangle - 2\langle (y - f_{\text{quad}})\, f_{\text{lin}} \rangle + \langle f_{\text{lin}}^2 \rangle$$

The cross-term vanishes: $f_{\text{lin}} = \alpha \bar{w} \cdot x$ is a linear function of x, while $y - f_{\text{quad}}$ is a quadratic form $x^\top (\cdots) x$. Their product is a degree-3 polynomial in x, and all odd-degree moments of an isotropic Gaussian are zero. Therefore $\mathcal{L} = \langle (y - f_{\text{quad}})^2 \rangle + \langle f_{\text{lin}}^2 \rangle$.

Term C: $\langle f_{\text{lin}}^2 \rangle = \alpha^2 \langle (\bar{w} \cdot x)^2 \rangle = \alpha^2 \sum_{ij} \bar{w}_i \bar{w}_j \langle x_i x_j \rangle = \alpha^2 \sum_i \bar{w}_i^2 \cdot \frac{1}{D} = \frac{\alpha^2}{D} |\bar{\boldsymbol{w}}|^2$.

(b) We have $y - f_{\text{quad}} = \frac{1}{2} x^\top A\, x$ with $A = \beta_\star \beta_\star^\top - \alpha\epsilon M$. So:

$$\langle (y - f_{\text{quad}})^2 \rangle = \frac{1}{4}\langle (x^\top A\, x)^2 \rangle = \frac{1}{4D^2}\left[(\operatorname{Tr} A)^2 + 2|A|_F^2 \right]$$

This gives:

$$\text{Term A} = \frac{1}{4D^2}(\operatorname{Tr} A)^2 = \frac{1}{4}\left(\frac{|\beta_\star|^2 - \alpha\epsilon \operatorname{Tr} M}{D} \right)^2$$

$$\text{Term B} = \frac{1}{2D^2}|A|_F^2 = \frac{1}{2D^2}|\alpha\epsilon\, M - \beta_\star \beta_\star^\top|_F^2$$

(c) - **Term A (variance error)**: Measures whether the overall *scale* of the learned quadratic form ($\alpha\epsilon \operatorname{Tr} M$) matches the target's scale ($|\beta_\star|^2$). - **Term B (misalignment error)**: Measures whether the *shape* (directional structure) of M is aligned with the rank-1 target $\beta_\star\beta_\star^\top$ - this is the term most closely related to NTK alignment / CKA. - **Term C (linear power)**: Penalises any energy the network puts into linear functions of the input, since the target is purely quadratic and contains no linear component.

(d) At initialisation, $\bar{w} = 0$ and $M = I$: - **Term C = 0** (no linear power yet). - **Term A** depends on $(|\beta_\star|^2 - \alpha\epsilon D)^2/D^2$, which is generically nonzero. - **Term B is large**: the identity matrix I is spread uniformly across all D directions, while $\beta_\star\beta_\star^\top$ is rank-1. The Frobenius distance $|\alpha\epsilon I - \beta_\star\beta_\star^\top|_F$ is large.

In early training, the network needs to reduce train loss. From Problem 2(b), the initial kernel's sensitivity to linear structure (scaling as $1/\sqrt{D}$) is much stronger than its sensitivity to quadratic structure (scaling as ϵ^2/D). So the network first exploits the linear component of the activation $\varphi(h) = h + \frac{\epsilon}{2}h^2$: it grows $\bar{w}$ to produce a linear function $\alpha\bar{w} \cdot x$ that approximately fits the training data. This *reduces train loss* but *increases Term C* on the test set - the network is fitting a linear function to a quadratic target, which works on finite training data but fails to generalise. Meanwhile Term B (misalignment) remains large because no feature learning has occurred. This is the memorisation phase.

Problem 4 — The Role of α: Controlling Laziness (10 min)

Recall the model $f(\boldsymbol{w}, \boldsymbol{x}) = \frac{\alpha}{N}\sum_{i=1}^{N} \varphi(\boldsymbol{w}_i \cdot \boldsymbol{x})$ with target $y(\boldsymbol{x}) = \frac{1}{2}(\boldsymbol{\beta}_\star \cdot \boldsymbol{x})^2$. The parameter α multiplies the network output but does **not** appear in the target. We train with gradient flow on the MSE loss $\mathcal{L} = \frac{1}{2}\sum_\mu (y_\mu - f_\mu)^2$.

Important note. In the paper, large-α experiments are implemented using the centered predictor

$$\tilde{f}(\boldsymbol{w}, \boldsymbol{x}) = f(\boldsymbol{w}, \boldsymbol{x}) - f(\boldsymbol{w}_0, \boldsymbol{x})$$

and the learning rate is rescaled as $\eta \propto \alpha^{-2}$. Here we give the basic scaling intuition in raw gradient-flow time.

(a) Scaling of the NTK. Compute the Jacobian $\nabla_{\boldsymbol{w}_i} f$ and show that it scales as α. Deduce that the NTK Gram matrix $(K_0)_{\mu\nu} = \sum_i \nabla_{\boldsymbol{w}_i} f(\boldsymbol{x}_\mu) \cdot \nabla_{\boldsymbol{w}_i} f(\boldsymbol{x}_\nu)$ scales as α^2.

(b) Kernel regression timescale. In the linearized (lazy) regime, the prediction dynamics are $\dot{f}_\mu = -\sum_\nu K_{\mu\nu}(f_\nu - y_\nu)$. Argue that the timescale for the training predictions to converge to the targets is:

$$t_{\text{kernel}} \sim \frac{1}{\alpha^2}$$

Note that the targets $y_\mu = \frac{1}{2}(\boldsymbol{\beta}_\star \cdot \boldsymbol{x}_\mu)^2$ are $O(1)$ — they are independent of α.

(c) Parameter displacement. The linearized model is $f_\mu \approx f_\mu^0 + \sum_i \nabla_{\boldsymbol{w}_i} f|_{\boldsymbol{w}_0} \cdot \delta \boldsymbol{w}_i$, where $\delta \boldsymbol{w}_i = \boldsymbol{w}_i - \boldsymbol{w}_i(0)$. To fit the training data, the network needs $\delta f_\mu \sim O(1)$. Using the fact that $\nabla_{\boldsymbol{w}_i} f \sim \alpha$, show that the required parameter displacement satisfies:

$$|\delta \boldsymbol{w}| \sim \frac{1}{\alpha}$$

Explain why this means large α keeps the network in the lazy regime.

(d) Feature learning timescale. The NTK depends on the parameters through $\varphi'(\boldsymbol{w}_i \cdot \boldsymbol{x})$. Show that the fractional change in the NTK during kernel regression is:

$$\frac{\Delta K}{K_0} \sim \frac{|\delta \boldsymbol{w}|}{|\boldsymbol{w}_0|} \sim \frac{1}{\alpha}$$

(using $|\boldsymbol{w}_0| \sim O(1)$ at standard initialization). Deduce that for large α, the NTK has barely rotated by the time train loss reaches zero: memorization happens **before** any significant feature learning.

For the NTK to rotate by $O(1)$ (which is what generalization requires), the parameters must change by $O(1)$. This happens on a much longer timescale than the initial kernel fit. A standard scaling summary is:

$$t_{\text{feature}} \sim \alpha^2$$

(e) The grokking gap. Compute the ratio $t_{\text{feature}}/t_{\text{kernel}}$ as a function of α. Describe what happens in the two limits $\alpha \to 0$ and $\alpha \to \infty$.

Solution 4

(a) We have $f(\boldsymbol{w}, \boldsymbol{x}) = \frac{\alpha}{N} \sum_i \varphi(\boldsymbol{w}_i \cdot \boldsymbol{x})$, so:

$$\nabla_{\boldsymbol{w}_i} f = \frac{\alpha}{N} \varphi'(\boldsymbol{w}_i \cdot \boldsymbol{x})\, \boldsymbol{x}$$

This scales as α (everything else — φ', $\boldsymbol{x}$, $1/N$ — is independent of α). The NTK Gram matrix is:

$$K_{\mu\nu} = \sum_i \nabla_{\boldsymbol{w}_i} f(\boldsymbol{x}_\mu) \cdot \nabla_{\boldsymbol{w}_i} f(\boldsymbol{x}_\nu) = \frac{\alpha^2}{N^2} \sum_i \varphi'(\boldsymbol{w}_i \cdot \boldsymbol{x}_\mu)\, \varphi'(\boldsymbol{w}_i \cdot \boldsymbol{x}_\nu)\, (\boldsymbol{x}_\mu \cdot \boldsymbol{x}_\nu)$$

This scales as α^2. $\square$

(b) The linearized prediction dynamics $\dot{f}_\mu = -\sum_\nu K_{\mu\nu}(f_\nu - y_\nu)$ describe exponential convergence at a rate set by the eigenvalues of K_0. Since $K_0 \sim \alpha^2$, its eigenvalues are $O(\alpha^2)$, and the convergence time is:

$$t_{\text{kernel}} \sim \frac{1}{\lambda_{\min}(K_0)} \sim \frac{1}{\alpha^2}$$

The targets $y_\mu = \frac{1}{2}(\boldsymbol{\beta}_\star \cdot \boldsymbol{x}_\mu)^2$ involve only the fixed teacher $\boldsymbol{\beta}_\star$ and the data $\boldsymbol{x}_\mu$, neither of which depends on α. So $y_\mu = O(1)$ regardless of α. The kernel regression timescale **decreases** with α: larger output scale means faster memorization. $\square$

(c) The linearized model gives $\delta f_\mu = \sum_i \nabla_{\boldsymbol{w}_i} f \cdot \delta \boldsymbol{w}_i$. Since $\nabla_{\boldsymbol{w}_i} f \sim \alpha$, the total change is schematically $\delta f \sim \alpha \cdot |\delta \boldsymbol{w}|$. To fit the data we need $\delta f \sim O(1)$. Therefore:

$$\alpha \cdot |\delta \boldsymbol{w}| \sim O(1) \qquad \Longrightarrow \qquad |\delta \boldsymbol{w}| \sim \frac{1}{\alpha}$$

Large α means tiny parameter displacement. The linearization $f(\boldsymbol{w}) \approx f(\boldsymbol{w}_0) + \nabla f \cdot (\boldsymbol{w} - \boldsymbol{w}_0)$ is accurate when $|\boldsymbol{w} - \boldsymbol{w}_0|$ is small relative to the scale over which ∇f varies, which is $O(1)$ (set by the curvature of φ). Since $|\delta \boldsymbol{w}| \sim 1/\alpha \ll 1$ for large α, the linearization holds throughout training: the network stays lazy. $\square$

(d) The NTK depends on parameters through $\varphi'(\boldsymbol{w}_i \cdot \boldsymbol{x})$. A parameter change $\delta \boldsymbol{w}_i$ changes the argument by $\delta \boldsymbol{w}_i \cdot \boldsymbol{x} \sim |\delta \boldsymbol{w}| \cdot |\boldsymbol{x}|$. At standard initialization, $\boldsymbol{w}_i \cdot \boldsymbol{x} \sim O(1)$, so the fractional change in φ' is:

$$\frac{\Delta \varphi'}{\varphi'} \sim \frac{\varphi''(\boldsymbol{w}_i \cdot \boldsymbol{x})\, \delta \boldsymbol{w}_i \cdot \boldsymbol{x}}{\varphi'(\boldsymbol{w}_i \cdot \boldsymbol{x})} \sim |\delta \boldsymbol{w}| \sim \frac{1}{\alpha}$$

Since $K \propto (\varphi')^2$, the fractional NTK change is also $\Delta K / K_0 \sim 1/\alpha$.

For large α, this is small: by the time the network has memorized the training data, the NTK has changed by only a fraction $\sim 1/\alpha$ from its initial value. The features are essentially frozen. Memorization is complete, but no feature learning has occurred — the kernel is still misaligned with the task.

For the NTK to align with the target (rotating $\boldsymbol{M}$ toward $\boldsymbol{\beta}_\star \boldsymbol{\beta}_\star^\top$), we need $\Delta K / K \sim O(1)$, which requires $|\delta \boldsymbol{w}| \sim O(1)$. This takes much longer than the initial lazy fit, giving the scaling summary

$$t_{\text{feature}} \sim \alpha^2 \qquad \square$$

(e) The grokking gap is:

$$\frac{t_{\text{feature}}}{t_{\text{kernel}}} \sim \frac{\alpha^2}{1/\alpha^2} = \alpha^4$$

This grows as the **fourth power** of α. The two limits:

- $\alpha \to 0$: The linearization is never valid — parameters change by $O(1)$ immediately, and feature learning happens from the start. Train and test loss decrease together. **No grokking.**
- $\alpha \to \infty$: $t_{\text{kernel}} \to 0$ and $t_{\text{feature}} \to \infty$. The network instantly memorizes via kernel regression with the (misaligned) initial NTK, then takes forever to learn features. Train loss drops immediately; test loss plateaus at the poor kernel regression error. **Grokking never completes** — the network is permanently lazy. □

Problem 5 - The Role of ϵ: Initial Kernel Alignment (5 min)

Large ϵ means the initial kernel already places significant power on quadratic functions. Explain why this *reduces* or *eliminates* grokking. Conversely, why does small ϵ *increase* the grokking delay but lead to *lower* final test loss?

Solution 5

Large ϵ: recall from Problem 2(b) that the initial kernel's sensitivity to quadratic structure scales as ϵ^2/D. When ϵ is large, this quadratic sensitivity is comparable to the linear sensitivity $(1/\sqrt{D})$, so the initial kernel already "sees" the quadratic target almost as well as it sees linear functions. This means kernel regression at initialisation can fit the target reasonably well without needing to change features - the network generalises early and there is no grokking gap.

Small ϵ: the initial kernel is nearly blind to quadratic structure (its quadratic sensitivity $\epsilon^2/D \ll 1/\sqrt{D}$). The kernel regression solution at initialisation fits the training data primarily through linear components (growing $\bar{w}$, increasing Term C), which fail to generalise. The network *must* eventually learn features (rotate M toward $\beta_\star\beta_\star^\top$) to generalise, and this takes time - producing a long grokking delay. However, once features are learned, the network finds a solution specialised to the target, which can achieve lower test error than the generic kernel solution. Bad initial features force more feature learning, resulting in a better final solution.

Problem 6 - Synthesis: Why Grokking is a Lazy-to-Rich Transition (5 min)

(a) State the three conditions identified by Kumar et al. that are jointly sufficient for grokking.

(b) Using the loss decomposition from Problem 3, describe the two-phase dynamics during grokking: - Phase 1 (early training): What happens to Terms A, B, C? - Phase 2 (late training): What happens to Terms A, B, C?

(c) Explain in 2-3 sentences why weight decay can help produce grokking but is not necessary, and how the lazy-to-rich framework subsumes weight-decay-based explanations.

Solution 6

(a) The three conditions are: 1. The initial NTK is *misaligned* with the target (small ϵ, or low CKA). 2. The dataset size is in a *Goldilocks zone*: large enough for generalisation to be possible, but small enough that train and test loss don't track each other. 3. The network starts in the *lazy regime* (large α), so feature learning is delayed.

(b) - **Phase 1** (memorisation): The network uses the linear component of φ to approximately fit the training data via kernel regression. Term C increases (linear power grows), Term B stays large (no alignment), Term A may fluctuate. Train loss decreases, but test loss stays high or even increases. - **Phase 2** (generalisation/grokking): Feature learning kicks in. M rotates toward $\beta_\star\beta_\star^\top$ (Term B decreases), $\bar{w}$ is driven back to zero (Term C decreases), and Term A adjusts as $\mathrm{Tr}M$ calibrates. Test loss finally drops.

(c) Weight decay is not necessary for grokking: the paper gives examples with zero weight decay and even increasing weight norm. But weight decay can help because it pushes the model out of the lazy regime by forcing the kernel to evolve, which encourages feature learning. The lazy-to-rich framework is thus more fundamental: weight decay is one of several ways to control the transition speed, not the root cause.